

-- Stored procedure, triggers, events, transactions, lock, ACID properties, grant, revoke, cte.

-- Stored procedure:-

-- Stored procedure is a precompiled SQL program stored inside the database that can be

-- executed whenever needed.

-- Instead of writing the same SQL queries repeatedly in application, you can store the logic

-- inside the database

-- Why are stored procedure used:-

-- 1) Reusability:-

-- write once, use many times

-- 2) Performance :-

-- Stored procedures are compiled and stored in MySQL,

-- so execution can be faster.

-- 3) Security :-

-- users can run procedures without direct table access.

-- 4) Reduce network traffic:-

-- instead of sending many queries, you can send one procedure call.

-- Syntax

DELIMITER \$\$

CREATE PROCEDURE PROCEDURE_NAME()

BEGIN

SQL STATEMENTS 1;

SQL STATEMENTS2;

SQL STATEMENTS3;

END \$\$

DELIMITER ;

-- SEMICOLON IS THE DEFAULT DELIMITER

-- DELIMITER IS USED BECAUSE MySQL NORMALLY ENDS A COMMAND WITH ;

-- BUT STORED PROCEDURES CONTAIN MANY ;

-- SO WE TEMPORARILY CHANGE THE DELIMITER.

use amazon;

select * from customers where customer_id = 101;

call searchCustomer(101);

```
-- IF PROCEDURE CONTAINS ONE QUERY. YOU DON'T NEED TO CHANGE THE DELIMITER AND YOU
DON'T NEED
-- TO SPECIFY BEGIN AND END.
CREATE PROCEDURE SHOW_CUSTOMERS()
SELECT * FROM CUSTOMERS;
```

```
-- HOW TO CALL A PROCEDURE
-- CALL PROCEDURE_NAME();
```

```
CALL SHOW_CUSTOMERS();
```

```
-- CREATE A PROCEDURE WITH CUSTOMERS NAME AND WHICH PRODUCT THEY BOUGHT AND I ALSO
NEED EMPLOYEES TABLE
```

```
DELIMITER $$
CREATE PROCEDURE SHOW_TABLES()
BEGIN
SELECT *
FROM CUSTOMERS C
JOIN CUSTOMER_PRODUCTS CP
ON C.CUSTOMER_ID = CP.C_ID
JOIN PRODUCTS P
ON P.PRODUCT_ID = CP.P_ID;

SELECT * FROM EMPLOYEES;
END $$
DELIMITER ;
```

```
-- HOW TO DROP A PROCEDURE
DROP PROCEDURE SHOW_TABLES;
```

```
CALL SHOW_TABLES();
-- PROCEDURES WITH PARAMETERS:-
-- THERE ARE 3 TYPES OF PARAMETERS
-- 1) IN --> INPUT PARAMETER
-- 2) OUT --> OUTPUT PARAMETER
-- 3) INOUT --> BOTH INPUT AND OUTPUT PARAMETER
```

```
-- 1) IN :-
-- USED TO PASS VALUES TO PROCEDURE
SELECT * FROM CUSTOMERS
WHERE CUSTOMER_ID = 101;
```

```
DELIMITER //
CREATE PROCEDURE SEARCH_CUSTOMERS(IN CUST_ID INT)
BEGIN
    SELECT * FROM
    CUSTOMERS
```

```

        WHERE CUSTOMER_ID = CUST_ID;
END //
DELIMITER ;

DROP PROCEDURE SEARCH_CUSTOMERS;

CALL SEARCH_CUSTOMERS(104);

-- CREATE A PROCEDURE THAT SEARCHES ALL THE PRODUCTS THAT
-- HAVE A SPECIFIC STOCK AND PRICE.
DELIMITER $$
CREATE PROCEDURE SEARCH_PRODUCTS(IN P_STOCK INT, IN P_PRICE DECIMAL(9,2))
BEGIN
SELECT *
FROM PRODUCTS
WHERE STOCK >= P_STOCK AND PRICE >= P_PRICE;
END $$
DELIMITER ;

CALL SEARCH_PRODUCTS(50,70000);

-- OUT AND INOUT PARAMETER.
-- OUT:-
-- used to return a value from procedure.
delimiter $$
create procedure product_price(out avg_price decimal(9,2), out total_price
decimal(9,2))
begin
    select avg(price) into avg_price -- return 146000
    from products;
    select sum(price) into total_price -- return 733340
    from products;
end $$
delimiter ;

call product_price(@avg_price, @total_price);
select @avg_price;

call summary_table();

-- session variables
select @student;
-- global variables
show variables like 'profiling';

--

```

```

create database if not exists stored_procedureDB;
use stored_procedureDB;

create table orders(
order_id int primary key auto_increment,
customer_name varchar(225),
order_date date
);

DELIMITER $$
CREATE PROCEDURE CREATE_ORDERS(IN CUST_NAME VARCHAR(225),
                                OUT LAST_ORDER_ID
INT)
BEGIN
    INSERT INTO ORDERS (CUSTOMER_NAME,ORDER_DATE)
    VALUES (CUST_NAME,curdate());
    SET LAST_ORDER_ID = last_insert_id();
END $$
DELIMITER ;

CALL CREATE_ORDERS("Sonal",@last_order_id);

select @last_order_id;

select * from orders;

-- inout :-
select * from orders;

delimiter $$
create procedure total_order_cost(inout customer_details int)
begin
    select sum(total_amount) into customer_details
    from orders
    where c_id = customer_details;
end $$
delimiter ;

set @customer_details = 102;
call total_order_cost(@customer_details);
select @customer_details;

select sum(total_amount) from orders where c_id = 102;

```

